

CS F342 Comp Arch — Midsem Cheatsheet
(2021-2025 papers)

Paper	Question → Item #
21-22	1a→5, 1b→2, 1c→3, 1d→4, 1e→29, 2a→19, 2b→30, 2c→18, 3→31
22-23	1a→1, 1b→2, 1c→4, 1d→18, 2a→23, 2b→17, 2c→24, 2d→25, 2e→26, 3→36, 4→27, 5→33, 6→28
23-24	1a→6, 1b→9, 1c→7, 1d→18, 1e→5, 1f→15, 2a→20, 2b→22, 3a→32, 3b→19, 4→39, 5a→17, 5b→34
24-25	1a→8, 1b→9, 1c→14, 1d→4, 1e→17, 1f→18, 1g→16, 2→37, 3a→21, 3b→32, 3c→38
25-26	1a→12, 1b→13, 1c→35, 2→36, 3a→9, 3b→4, 3c→10, 3d→11, 4a→14, 4b→18, 4c→16

Tog after each heading = papers where it appeared (yr-Q). Same Q again in exam → write the same answer. Opcodes: R 00000, lw 100011, sw 101011, beq 000100, addi 001000, j 000010, jal 000011. Multicycle cycles: lw, sw 4, R 4, addi 4, beq 3, j 3.

A. THEORY

- #1 Why opcode + funct fields instead of only opcode? 22-1a
- A 6-bit opcode can encode only $2^6=64$ instructions. All R-type instrs share opcode 000000 and are told apart by the 6-bit funct → 64 more ALU ops without widening the 32-bit instruction.
 - R-type needs no 16-bit immediate, so its low bits are free for shamt+funct; I/J types need those bits for imm/address, so they get unique opcodes. Keeps all instrs 32-bit with fields at fixed positions (regularity → simple hardware).
 - Decoding becomes two-level: main decoder looks only at opcode (6 inputs); ALU decoder uses funct+ALUOp → smaller, faster control (see #10).

#2 Why is single-cycle control called a "Truth Table"? 21-1b, 22-1b

In single-cycle MIPS every instr finishes in one clock, so the control unit is **purely combinational**: no state/memory, its outputs (RegWrite, RegDst, ALUSrc, Branch, MemWrite, MementoReg, ALUOp, Jump) depend *only* on the present inputs (opcode, funct). A combinational circuit is completely specified by listing outputs for every input combination = a truth table (one row per opcode), which is then implemented as SOP logic/PLA/ROM. (Multicycle control is an FSM → state table, not truth table.)

#3 How does the processor capture the side effects when instrs complete? 21-1c

Side effects = changes to architectural state: **register file**, **data memory** and **PC**. Combinational logic computes the new values during the cycle; they are captured only at the **rising clock edge** by state elements whose write enable is asserted: RF written if RegWrite=1 (WD3→arg A3), memory written if MemWrite=1 (WD→M[A]), PC←PC' every edge (multicycle: PCEn). Intermediate results in multicycle/pipeline are held in non-architectural registers (IR, MDR, A, B, ALUOut / pipeline regs) and committed in the write-back step.

#4 <<2 behaves as <<1 — which instrs affected? 21-1d, 22-1c, 24-1d, 25-3b

- Shift-by-2 converts a *word* offset to a *byte* offset; used only for PC-relative/Jump targets.
- beq/bne (tack)**: PCBranch = PC+4 + (SignImm<<1) = PC+4+2*imm instead of 4*imm → wrong, half-word aligned address (e.g., imm=3 → PC+4+6 not PC+4+12) → fetches garbage/misaligned instr. Not-taken branch still OK (PC+4).
- j (and jal)**: JTA = [PC+4][31:28], addrc<<1 → 26b<<1 = 27 bits only; target = half the intended address, wrong/misaligned → jumps to wrong location.
- Same in multicycle (ALUOut←PC+SignImm<<2 in Decode; jump path). **Not affected**: R-type (add, sub, and, or, slt), lw, sw, addi — they don't use the shifters (lw/sw offsets are already byte offsets).

#5 Stuck control signals (single-cycle) 21-1a, 23-1e

Signal	Malfunction & why
MementoReg#1	R-type (add,sub,and,or,slt) & addi : write ReadData (=M[ALUResult]) to reg instead of ALUResult. lw OK; sw, beq, j OK (RegWrite=0).
MementoReg stuck-at-0	lw only : writes ALUResult (the address RF[rs]+imm) into rt instead of loaded data. All others OK.
ALUOp#01	ALUOp 00→01 → ALU subtracts : lw, sw (address = rs+imm), addi (rs+imm) wrong, beq (01) OK. R-type: 10→11, ALU decoder looks at funct when ALUOp1=1 (11 treated as 1X) → OK (if 11 were undefined, R-type would also fail), j OK.
RegDst#0	R-type : result written to rt (20:16) instead of rd (15:11) → wrong reg & corrupts r. lw/addi OK (use rt); sw, beq, j OK (no write).

- #6 Can rt be the dest for R-type (and rd for others)? 23-1a
- Not advisable** - *no*. MIPS reads registers from fixed fields: Read reg1 = rs (25:21), Read reg2 = rt (20:16) for *all* instrs, so the RF can be read in **parallel with decoding**. sw and beq must read rt as a source; if R-type wrote rt, its 2nd source would be in rd, so a mux (controlled by decoder) would be needed on **Read register 2** → RF read starts only after control decode → longer critical path ($t_{\text{ctrl}}+t_{\text{mux}}$). Vice-versa (I-type writing rd) is impossible: bits 15:11 are part of the 16-bit immediate in lw/addi. Hence dest mux (RegDst) is on the write side: rt for I-type, rd for R-type.

- #7 Do we need opcode in a special-purpose processor? 23-1c
- No**. A single-purpose processor executes one fixed algorithm; the algorithm is hard-coded in the controller FSM which directly sequences the datapath control signals each state. There is no program/instruction memory, no fetch and no decode, so no opcode. Only status inputs (e.g., comparator outputs) feed the controller. Opcode is needed only in general-purpose (programmable) processors.

#8 Single-cycle vs multi-cycle control unit 24-1a

Single-cycle	Multi-cycle
Combinational (truth table): signals = f(opcode,funct)	Sequential FSM: state reg + next-state logic + output logic; signals = f(state) (Moore), next state = f(state, opcode)
Signals constant for whole (long) cycle; CPI=1	Signals change every state; CPI 3-5 (long) cycle; CPI=1
Signals: RegDst, RegWrite, ALUSrc, Branch, MemWrite, MementoReg, ALUOp, Jump	Adds lOrD, lRWrite, PCWrite, ALUSrcA, ALUSrcB[1:0], PCSrc[1:0]; enables for PC/IR
Separate I/D memories & adders → no sequencing	Shared memory & ALU reused across states
Only hardwired	Hardwired FSM or microprogrammed (#9)

- #9 How are states generated: hardwired vs microprogrammed? 23-1b, 24-1b, 25-3a
- Hardwired**: current state kept in a **state register** (flip-flops, e.g., 4 bits for S0-S11). **Next-state logic** (gates/PLA) computes S' = f(S, opcode, Zero) and loads it at each clock edge; **output logic** decodes S to control signals. (Alternative: sequence counter + decoder giving timing states T0,T1,..., cleared at end of instr.)
- Microprogrammed**: each state is a **microinstruction** stored

in a control ROM; the state register is the **µPC** (address). Next state is generated by an address-select logic chosen by the µinstr's sequencing field: (i) µPC+1 (sequential), (ii) **dispatch ROM** indexed by opcode (Decode/MemAdr branching), (iii) Jump to 0 (fetch). Control signals are read directly from the µinstr bits. Easy to modify, slower; hardwired faster.

#10 Why two-level control (main decoder + ALU decoder)? 25-1c

Main decoder: opcode(6) → RegWrite, RegDst, ALUSrc, Branch, MemWrite, MementoReg, Jump, **ALUOp[1:0]**. ALU decoder: ALUOp(2)+funct(6) → ALUControl. Benefits: (1) **smaller logic**: a monolithic controller would have 12 inputs ($2^{12}=4096$ rows) vs 2^6+2^6 ; (2) faster & simpler to design/verify; (3) **modular**: adding an R-type instr only changes the ALU decoder; ALU decoder reused in multicycle & pipeline; (4) funct only matters for R-type, so it isn't routed into the main decoder.

#11 PC enable: single vs multi-cycle? 25-3d

Yes. Single-cycle: PC has **no enable** — every instruction completes in 1 cycle, so PC←PC' at every clock edge. Multicycle: an instr takes 3-5 cycles and the PC (memory address, PC+4, branch target computations) must hold its value during intermediate states, so PC has an enable: **PCEn = PCWrite & Branch-Zero** (PCWrite in Fetch & Jump states; Branch-Zero in beq state).

#12 Why can't single-cycle use one memory for fetch + data? 25-1a

A single-ported memory has one address port and one read/write data port → only one **access per cycle**. Single-cycle lw/sw need instruction fetch AND data read/write in the **same** cycle → structural hazard (address port conflict). **Fix**: separate **instruction memory** and **data memory** (Harvard; split l-cache/D-cache) — or a dual-ported memory; (multicycle instead reuses one memory in different cycles with the lOrD mux bits).

#13 Illegal control-signal combination 25-1b

e.g. **RegWrite=1 with MemWrite=1** (during sw): memory is written correctly but the RF also writes Result (ALUResult = address, or garbage ReadData) into register rt/rd (whatever RegDst selects) → the source register (rt) being stored gets **corrupted**. Others: Branch=1 & Jump=1 (two PC sources, Jump mux overrides → wrong PC); MementoReg=1 with MemRead=0 (writes undefined data).

#14 Methods of generating control signals in pipeline 24-1c, 25-4a

- Centralised (decode once in ID)**: same control unit as single-cycle generates all signals in ID; they travel with the instr through ID/EX/EX/MEM, MEM/WB; each stage uses its group (EX: RegDst, ALUSrc, ALUControl; MEM: Branch, MemWrite; WB: RegWrite, MementoReg). Adv: one decoder, reuses single-cycle design, simple, consistent. Disadv: wider pipeline registers (more flip-flops, power).
- Distributed (decode per stage)**: pass opcode/funct down the pipe, each stage decodes its own signals. Adv: narrower pipeline regs. Disadv: duplicated decoders (area) and decode delay inside each stage's critical path.
- Microprogrammed/ROM per stage**: flexible, easy to change; slower, big ROM. Hazard unit separately generates Stall, Flush, Forward signals.

#15 Can forwarded data be taken before the stage (pipeline) registers? (R-type→R-type) 23-1f

No. It computes its result in EX in cycle c; I2 needs it in its EX in cycle c+1. In cycle c+1 the wire before EX/MEM carries I2's *own* ALU output (I1's value is now in EX/MEM) → wrong data / combinational loop. Taking it in cycle c would chain ALU → forward mux → ID path in one cycle (longer clock). The pipeline register holds I1's result stable for the whole next cycle, so forward from **EX/MEM** (ALUOutM) and **MEM/WB** (ResultW) registers: ForwardAE=10 if (rsE=0 & rsE==WriteRegM & RegWriteM), 01 if from WB.

- #16 Setup & hold time; derive clock constraints 24-1g, 25-4c
- Setup t_{setup}** : min time D must be stable before the clock edge. **Hold t_{hold}** : min time D must stay stable after the edge. Path R1 → combinational logic → R2:
- Setup**: data launched at edge 0 arrives at R2 latest at $t_{\text{pcq}}+t_{\text{pd}}$; must arrive t_{setup} before next edge: $T_c \leq t_{\text{pcq}} + t_{\text{pd}} + t_{\text{setup}} \Rightarrow t_{\text{pd}} \leq T_c - (t_{\text{pcq}}+t_{\text{setup}})$ (sequencing overhead).
- Hold**: new data launched at the same edge must not reach R2 before t_{hold} ; earliest arrival $t_{\text{ccq}}+t_{\text{cd}}$; $t_{\text{ccq}} + t_{\text{cd}} \leq t_{\text{hold}}$ (independent of T_c ; fix by adding buffers). With clock skew: $T_c \geq t_{\text{pcq}}+t_{\text{pd}}+t_{\text{setup}}+t_{\text{skew}}+t_{\text{ccq}}+t_{\text{cd}} \leq t_{\text{hold}}+t_{\text{skew}}$ (pcq/pd = propagation = max; ccq/cd = contamination = min).

#17 128 registers & instruction set x4 — effect on fields 22-2b, 23-5a, 24-1e

Reg fields: $\log_2 128 = 7$ **bits** (was 5). 4x instrs → opcode **6→8 bits** (and funct 6→8 if R-types also x4).

R-type: op 8 + rs 7 + rt 7 + rd 7 + shamt 5 + funct 8 = **42 bits** (+10). **I-type**: op 8 + rs 7 + rt 7 + imm 16 = **38 bits** (+6). If instruction must stay 32-bit: I-type imm shrinks to 32-8-14 = **10 bits** (smaller constants/branch range); R-type has only 32-8-21 = **3 bits** left for shamt+funct → must drop shamt/shrink funct or use more opcodes. Effects: bigger/slower RF, bigger decoder, fewer bits for constants.

B. NUMERICALS / TRACING

#18 5-stage pipeline: regs written/read in the 5th cycle 21-2c, 22-1d, 23-1d, 24-1f, 25-4b

Cycle 5: I1 in **WB**, I2 MEM, I3 EX, I4 in **ID**, I5 IF. RF is **written in 1st half** of cycle (by I1) and **read in 2nd half** (by I4, which reads its rs & rt fields).

Program (I1...I5)	Written	Read (by I4)
21: or R2,R4,R5 / sw R5,R2(R0) / sub R0,R1,R2 / lw R3,R5(R1) / addi	R2 (or R1 (base; R3 on RD2 unused)	
22: or \$t2,\$t4,\$t5 / sub / sw / addi \$t1,\$t1,50 / lw	\$t2	\$t1
23: 24: or \$t2,\$t4,\$t5 / sw / addi / lw \$t3,\$t5(\$t1) / sub	\$t2	\$t1 (\$t3 unused)
25: addi \$t1,\$t1,50 / lw / or / sw \$t5,R2(\$t0) / sub	\$t1	\$t0, \$t5

I1	c1 c2 c3 c4 c5
I2	IF ID EX MEM WB — writes (1st half)
I3	IF ID EX MEM
I4	IF ID EX
I5	IF ID — reads (2nd half)

- #19 Cycles & CPI of loop program (single/multi-cycle) 21-2a, 23-3b
- R1=0, R2=10; loop adds 2 to R1 → beq evaluated at R1=0,2,4,6,8 (not taken) and 10 (taken) ⇒ beq x6, loop body (3 addi + j) x5.
- Instr count = 2 addi + 6 beq + 15 addi + 5 j = **28**.
- Multicycle** (addi 4, beq 3, j 3): $17 \times 4 + 6 \times 3 + 5 \times 3 = 68+18+15 =$

- 101 cycles**: CPI = $101/28 = 3.61$.
- Single-cycle**: 28 cycles, CPI = **1** (but long clock).

Unit	Inputs → Output
A1 (PC+4)	0x00400000; 4 → 0x00400004
SignExt / <<2	0x248C → 0x0000248C → 0x00009230
A2 (branch)	0x00400004 & 0x00009230 → 0x00409234
RF	A1=rs=0, A2=rt=0 → RD1=0, RD2=0
M2 RegDst	in0 = 00000 (rt), in1 = 00100 (rd) → X (RegWrite=0)
M3 ALUSrc	in0 = RD2 = 0, in1 = 0x0000248C → X
ALU decoder	ALUOp=XX, funct=001100 → ALUControl = XXXX (if ALUOp=00: 0010 add)
ALU	SrcA=RD1=0, SrcB=M3 out → result unused (X); Zero=X
Data mem	MemWrite=0 → not written; RD unused
M5 MementoReg	in0 ALUResult, in1 ReadData → X (not written)
M4 PCsrc	Branch-Zero=0 → in0 = 0x00400004
M1 Jump	in0 = 0x00400004, in1 = [PC+4][31:28]=0000, 0x0000248C, 00] = 0x00009230; Jump=1 → PC' = 0x00009230

#21 Multicycle fetches 0x1000248C — values per state (M0-M6 of given fig) 24-3a

0001 00|00 000|0 0000|0010 0100 1000 1100 → op=000100 = **beq** 50, **0x0248C** (rs=rt=0, imm=0x248C; rd bits=00100). Assume PC=0x00400000. 3 states:

State	Signals	Values
S0 Fetch	lOrD=0, lRWrite=1, ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCSrc=00, PCWrite=1, Jump=0	M0: Adr=PC=0x00400000; Mem RD=0x1000248C→lR, M4=PC, M5=4, ALUCtrl=0010 → ALU=0x00400004; M6→M1 → PC=0x00400004 (also ALUOut)
S1 Decode	ALUSrcA=0, ALUSrcB=11, ALUOp=00	RF A1=A2=0 → A=0, B=0; SignImm=0x0000248C, <<2=0x00009230; M4=0x00400004, M5=0x00009230 → ALU=0x00409234 → ALUOut. M2 in: 0/4 (unused)
S8 Branch	ALUSrcA=1, ALUSrcB=00, ALUOp=01, Branch=1, PCSrc=01	M4=A=0, M5=B=0, ALUCtrl=0110 → 0=0=0, Zero=1 ⇒ PCEn=Branch-Zero=1; M6=ALUOut → M1(Jump=0) → PC=0x00409234 (branch taken)

M3 (MementoReg) unused: RegWrite=0 throughout.

#22 Latencies (Table-1) & minimum clock period 23-2b

mem 250, RF 150, mux 25, ALU 200, adder 150, gate 5, clk-q 30, setup 20, sext 50, control 50.

Instr	Critical path	ps
R-type	$t_{\text{pcq}}+t_{\text{mem}}+t_{\text{rf}}+t_{\text{mux}}(\text{ALUSrc})+t_{\text{ALU}}+t_{\text{mux}}(\text{MementoReg})+t_{\text{setup}}$	700
LW	$30+250+\text{max}(\text{RF } 150, \text{sext}+\text{mux } 75)+200+250(\text{Dmem})+25+20$	925
SW	$30+250+150+200+250(\text{Dmem write})$	880
BEQ	$30+250+150+25+200+5(\text{AND})+25(\text{M4})+25(\text{M1})+20(\text{PC setup})[\text{adder path } 30+250+50+150=480 \text{ not critical}]$	730

Control (30+250+50=330 ps) is ready before RF data (430) → not on critical path. $t_{\text{cmin}} = \text{max} = 925 \text{ ps (lw)} \Rightarrow f = 1.08 \text{ GHz}$.

#23 Bytes M3=00100000, M2=01000000, M1=00001000, M0=00000001 → instr 22-2a

MIPS big-endian (M0 = MSB): 0000 0001 0000 1000 0100 0000 0010 0000 = 0x01084020 → 000000|01000|01000|01000|00000|100000 → **R-type: add \$t0, \$t0, \$t0** (op 0, rs=rt=rd=8, shamt 0, funct 0x20). (If little-endian, M3 = MSB: 0x20400801 = 001000|000010|00000|00001000000000001 ⇒ I-type addi \$zero, \$v0, 2049.)

#24 PC = 0x300000000: range reachable by beq 22-2c

Target = PC+4 + SignExt(imm)-4, imm 16-bit ∈ [-2¹⁵, 2¹⁵-1]. PC+4 = 0x300000004.

Min = 0x300000004 - 2¹⁷ = 0x300000004 - 0x200000 = **0x2FFE0004**

Max = 0x300000004 + (2¹⁵-1)-4 = 0x300000004 + 0x1FFFC = **0x30020000**

= ±128 KB (-32768 ... +32767 instructions from PC+4), word aligned.

#25 Horizontal µprogram control-memory width 22-2d

Horizontal: one bit per control line → control field = **11 bits**. Condition select for 8 status bits: $\log_2 8 = 3$ bits (MUX select). Next-address for 128 words: $\log_2 128 = 7$ bits.

Width = 11 + 3 + 7 = **21 bits**; control memory = 128 × 21 bits. (If status were also one-hot: 11+8+7 = 26.)

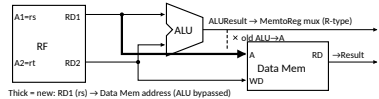
#26 Translate MIPS to C (D base \$s6, m=\$s0) 22-2e

addi \$t0,\$s6,4	# t0 = &D[1]
add \$t1,\$s6,\$0	# t1 = &D[0]
sw \$t1,0(\$t0)	# D[1] = &D[0]
lw \$t0,0(\$t0)	# t0 = D[1] (= &D[0])
add \$s0,\$t1,\$t0	# m = &D[0] + D[1]
add \$s0,\$s0,\$t0	# m = m + D[1]

C: D[1] = (int)&D[0]; m = (int)&D[0] + D[1] + D[1]; ⇒ m = 3-(base address of D).

#27 lD/sl (no offset) in single-cycle 22-4

- (a) Old lw: 30+250+150+200+250+25+20 = 925 ps. New lD needs no ALU: $t_{\text{pcq}}+t_{\text{mem}}+t_{\text{rf}}+t_{\text{mem}}+t_{\text{mux}}+t_{\text{RFwrite}} = 30+250+150+250+25+20 = 725$. R-type = 30+250+150+250+200+25+20 = 700; beq = 700. ⇒ **$T_c = 725 \text{ ps}$** (lD still longest).
- (b) Each lw/sw now needs an extra addi for the address: N' = N(1+0.26+0.12) = 1.38N. Old: N×925 = 925N. New: 1.38N×725 = 1000.5N ⇒ **slower**; 1000.5/925 = 1.082 ⇒ **≈ 8.2% slower**.
- (c) The fraction f of loads/stores that need an extra address instruction) vs the cycle-time gain: faster iff (1+f)·725 < 925 ⇒ **f < 27.6%**. (Here f = 38% → slower.)
- (d) Data-memory address taken directly from RD1:

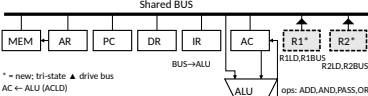


Thick = new: RD1 (rs) → Data Mem address (ALU bypassed)

C. DESIGN / DATAPATH MODIFICATIONS

#29 Simple CPU (shared bus): add OR R1,R2 (AC ← R1 V R2) 21-1e

(1) Min components: registers R1, R2 (if absent) with load (R1LD, R2LD) and tri-state buffers onto the bus (R1BUS, R2BUS); ALU gets an OR function (ALU_OR). Single bus ⇒ two operands can't travel together, so AC is used as temp: AC←R1, then AC←ACVR2.

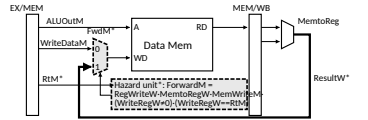


(2) Modified: ALU (OR op + ALU_OR input), control unit (decode new opcode; 7 instrs ⇒ 3-bit opcode, new states OR1, OR2, new signal equations), AC load logic, bus (R1, R2 drivers).

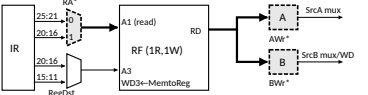
(3) States / μops: FETCH1: AR←PC; FETCH2: DR←M, PC←PC+1; FETCH3: IR←DR[op], AR←DR[addr]; OR1: AC←R1 (R1→bus, ALU pass); OR2: AC←ACVR2 ⇒ FETCH1. OR1 = I_OR T3, OR2 = I_OR T4 (I_OR = opcode decoder output).

R1BUS = OR1; R2BUS = ALU_OR = OR2; ALU_OR = OR2; ALU_PASS = LD2 + OR1; CLD = LD2 + ADD2 + AND2 + OR1 + OR2 (old terms + new); ALU: seq. counter after OR2.

#30 Pipeline: hazard LW Rj,off(Ri) ⇒ SW Rj,off(Rk) 21-2b
sw needs Ri only in MEM (as write data); lw's data is ready at end of its MEM ⇒ forward MEM/WB ⇒ MEM (no stall). Add RTM (5 b) to EX/MEM and a mux before Dmem WD. Detect (ForwardM=1): RegWriteW · MemtoRegW = RTM. WriteDataM = ForwardM ? ResultW : WriteDataM. If base Rk = ri (Address needs the loaded value in EX) ⇒ load-use stall 1 cycle: lwstall = MemtoRegE · (RsD = RE); modify old lwstall so sw data reg doesn't stall: lwstall = MemtoRegE · ((RsD == RE) + (RtD == RE)) · MemWriteW; StallF = StallD = FlushE = lwstall. R0 never forwarded.



#31 Multicycle with 1-read-port RF 21-3
(a) Add a 2:1 MUX, 5-bit (inputs IR[25:21]=rs and IR[20:16]=rt, 1 select "RA", 5-bit output) on the single Read-register address. The single RD (32 b) feeds both A and B, so A and B get write enables (AW, BW). Modified: RF, A, B, controller (one extra state). (b)



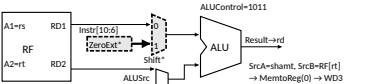
(c) FSM (unlisted signals = 0; RA: 0=rs, 1=rt):

State	μop	Signals	Next
S0 Fetch	IR←M[PC]; PC←PC+4	lorD=0, IRWrite, ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCWrite	S1
S1 Decode-1	A←RF[rs]; ALUOut←PC+Slmm<2	RA=0, AW=1, ALUSrcA=0, ALUSrcB=11, ALUOp=00	R→S1b; lw→S2
S1b Decode-2	B←RF[rt]	RA=1, BW=1	S6
S6 Execute	ALUOut←A op B	ALUSrcA=1, ALUSrcB=00, ALUOp=10	S7
S7 ALU WB	RF[rD]←ALUOut	RegDst=1, MemtoReg=0, RegWrite	S0
S2 MemAdr	ALUOut←A+Slmm	ALUSrcA=1, ALUSrcB=10, ALUOp=00 (sw: also RA=1, BW)	S3
S3 MemRead	MDR←M[ALUOut]	lorD=1	S4
S4 Mem WB	RF[r]←MDR	RegDst=0, MemtoReg=1, RegWrite	S0

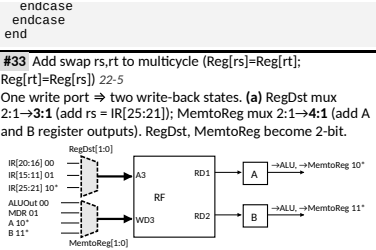
Optimized: lw needs only rs ⇒ skips S1b (still 5 cycles); R-type 5 cycles (+1). sw reads rt during S2 (port free).

#32 Add sra \$rd,\$rt,\$amt to single-cycle (+Verilog) 23-3a, 24-3b

rd ← rt >>> shamt (arith). Shift amount = Instr[10:6] must reach ALU; add a 2:1 mux on SrcA (RD1 / zero-ext shamt) with new signal Shift. ALU gets SRA op (ALUControl=1011): Result = \$signed(SrcB) >>> SrcA[4:0]. Main decoder: R-type row unchanged (RegWrite=1, RegDst=1, ALUSrc=0, MemtoReg=0, ALUOp=10). ALU decoder: ALUOp=10 & func=000011 (table: 100011) ⇒ ALUControl=1011, Shift=1. Other instrs: Shift=0 ⇒ unaffected.



```
module alu(input [31:0] a, b, input [3:0] f,
output reg [31:0] y, output zero);
always @(*) case (f)
4'b0000: y = a & b;
4'b0001: y = a | b;
4'b0010: y = a + b;
4'b0110: y = a - b;
4'b0111: y = ($signed(a) < $signed(b)) ? 1;
4'b1011: y = $signed(b) >>> a[4:0]; // SRA
default: y = 32'bX;
endcase
assign zero = (y == 32'b0);
endmodule
// new SrcA mux
assign SrcA = Shift ? {27'b0, Instr[10:6]} : RD1;
// ALU decoder (with sra added)
always @(*) begin
Shift = 0;
case (ALUOp)
2'b00: ALUControl = 4'b0010; // add
2'b01: ALUControl = 4'b0110; // sub
default: case (Func)
6'b100000: ALUControl = 4'b0010;
6'b100010: ALUControl = 4'b0110;
6'b100100: ALUControl = 4'b0000;
6'b100101: ALUControl = 4'b0001;
6'b101010: ALUControl = 4'b0111;
6'b000011: begin // sra
ALUControl = 4'b1011; Shift = 1; end
default: ALUControl = 4'bXXXX;
endcase
end
```



(b) Decode (S1) latches A=RF[rs], B=RF[rt]; op=SWAP ⇒ S12.

State	μop	RegDst	MemtoReg	RegWrite	Next
S12 Swap1	RF[rs]←B	10 (rs)	11 (B)	1	S13
S13 Swap2	RF[rt]←A	00 (rt)	10 (A)	1	S0

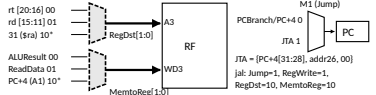
Other signals 0 (lorD, MemWrite, IRWrite, PCWrite, Branch = 0). A is re-latched at the same edge that writes RF[rs], so it still holds the old RF[rs] in S13 (IR unchanged). swap = 4 cycles.

#34 lwplus \$rt, imm(\$rs) in multicycle:
RF[r]←M[RF[rs]+imm]; RF[rs]←RF[rs]+4 23-5b
Yes, without changing the RF (1 write port): do the two writes in two different states. Datapath: RegDst mux 2:1→3:1 adding IR[25:21] (rs) as input 10 (same as #33 left mux). ALU computes rs+4 using the existing "4" input of ALUSrcB while memory is read.

State	μop	Signals (rest 0)	Next
S0, S1	Fetch, Decode (std)	A←RF[rs]	S2
S2 MemAdr	ALUOut←A+Slmm	ALUSrcA=1, ALUSrcB=10, ALUOp=00	S3
S3 MemRd+Inc	MDR←M[ALUOut]; ALUOut←A+4	lorD=1, ALUSrcA=1, ALUSrcB=01, ALUOp=00	S4
S4 WB1	RF[rt]←MDR	RegDst=0, MemtoReg=1, RegWrite=1	S5
S5 WB2	RF[rs]←ALUOut	RegDst=10, MemtoReg=0, RegWrite=1	S0

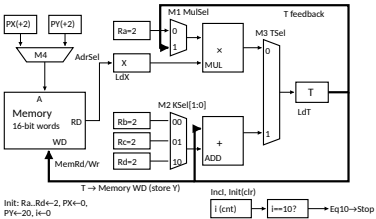
In S3 memory uses the old ALUOut (address) during the cycle; ALUOut is overwritten with A+4 only at the end edge. lwplus = 6 cycles.

#35 Add jal label to single-cycle 25-1c
jal: PC ← JTA = {PC+4[31:28], addr26, 00} (same as j via Jump mux M1) and sra(\$31) ← PC+4. Modifications: RegDst mux 2:1→3:1 (add constant 5'b11111 = 31); MemtoReg mux 2:1→3:1 (add PC+4 from address A1).



Main decoder row (op 000011): RegWrite=1, RegDst=10, ALUSrc=X, Branch=0, MemWrite=0, MemtoReg=10, ALUOp=XX, Jump=1. New control: RegDst and MemtoReg widened to 2 bits (equivalently one new signal Link/JAL driving two extra 2:1 muxes: WriteReg = JAL ? 31 : old; Result = JAL ? PC+4 : old). Existing instrs use 00/01 ⇒ unaffected. Return later with jr \$ra.

#36 16-bit multi-cycle single-purpose processor:
Yi=aXi³+bXi²+cXi+d, a=b=c=d=2, 0≤i<10 22-3, 25-2
Horner: Y = ((a·X + b)·X + c)·X + d ⇒ 3 multiplies + 3 adds using 1 multiplier + 1 adder over several cycles. 16-bit data, byte-addressable ⇒ pointers step by 2. X_{0..9} at addresses 0,2,...,18; Y_{0..9} at 20,...,38.

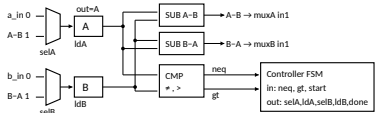


Init: Ra..Rd←2, PX←0, PY←20, i←0
Control signals (13): Init, LdX, LdT, MulSel, KSel[1:0] (2), Tsel, AdrSel, MemRd, MemWr, IncX, IncY, Incl. Status in: Eq10; output: Stop. Storage: Ra, Rb, Rc, Rd, X, T (16 b), PX, PY (16 b), i (4 b).

St	Operation	Asserted (rest 0)	Next
S0	Init: Ra..Rd←2, PX←0, PY←20, i←0	Init	S1
S1	If Eq10 ⇒ Stop; else X←M[PX], PX←PX+2	AdrSel=0, MemRd, LdX, IncX	Eq10? S9:S2
S2	T←a·X	MulSel=0, Tsel=0, LdT	S3
S3	T←T+b	KSel=00, Tsel=1, LdT	S4
S4	T←T·X	MulSel=1, Tsel=0, LdT	S5
S5	T←T+c	KSel=01, Tsel=1, LdT	S6
S6	T←T·X	MulSel=1, Tsel=0, LdT	S7
S7	T←T+d (=Y)	KSel=10, Tsel=1, LdT	S8
S8	M[PY]←T, PY←PY+2, i←i+1	AdrSel=1, MemWr, IncY, Incl	S1
S9	Done	Stop=1	S9

State reg 4 bits (10 states); total = 1 + 10×8 + 1 = 82 cycles. Clock period (25-2c): longest reg→reg path: Tc = tclk-q + max(tmux+tmul, tmux+tADD, tmux+tmem) + tmux+tsetup + tctrl
select lines settle after data). MUL dominates: Tc = tclk-q + tmux+tmul+tmux+tsetup; e.g. 30+25+400+25+20 = 500 ps (assumed tMUL=400). Memory path 30+25+250+20 = 325 ps.

#37 GCD in C, single-cycle datapath + controller, clock 24-2
int GCD(int a, int b){ while (a != b) { if (a > b) a = a - b; else b = b - a; } return a; }
(b) One loop iteration per clock. Datapath: regs A, B (ldA, ldB), 2:1 muxes (selA, selB) choosing input a/b or the difference, two subtractors (A-B, B-A), comparator (neq = A=B, gt = A>B), out = A.



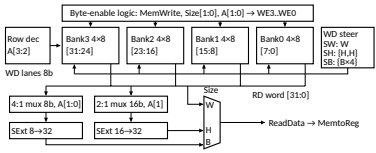
State	Cond	selA ldA selB ldB done	Next
S0 Load	—	0 1 0 1 0	S1
S1 Loop	neq=1, gt=1	1 1 X 0 0 (A←A-B)	S1
	neq=1, gt=0	X 0 1 1 0 (B←B-A)	S1
	neq=0	X X X 0 0	S2
S2 Done	—	X X X 0 1 (out=A)	S2/S0 on start

(c) Paths per cycle: data A, B ⇒ SUB ⇒ mux ⇒ reg; control A, B ⇒ CMP ⇒ FSM logic ⇒ mux select/load enable.

Tc = tclk-q + max(tsub, tcmp + tctrl) + tmux + tsetup. With Table-1 style values (clk-q 30, sub-cmp=add 150, control 50, mux 25, setup 20): 30 + max(150, 200) + 25 + 20 = 275 ps.

#38 Memory size & memory unit for SW/SH/SB/LW/LH/LB 24-3c

Size: 32-bit byte address ⇒ 2³² bytes = 4 GB = 2³⁰ words × 32 bits. Organise as 4 byte-wide banks (2³⁰×8 each); A[1:0] picks byte lane, A[31:2] picks row. For 16 byte locations: 4 banks × 4 rows × 8 b, row = A[3:2].



WE[3:0] (MemWrite=1): SW ⇒ 1111; SH ⇒ A[1:0]=0011, A[1]=1; 1100; SB ⇒ one-hot of A[1:0] (2:4 decoder). Reads: LW = word; LH = SExt16(half sel. by A[1]); LB = SExt8(byte sel. by A[1:0]). Control from opcode: Size[1:0] (00 W, 01 H, 10 B), MemWrite, MemRead.

Components & sizes (full 32-bit): 4 banks of 2³⁰×8; row decoder 30→2³⁰, 2:4 decoder + gates for WE; write-steer 3:1 mux 32 b; read: 4:1 mux 8 b, 2:1 mux 16 b, sign-ext 8→32 & 16→32, 3:1 mux 32 b. (16-loc version: 4 banks 4×8, 2→4 row decoder).

#39 Even number of 1s detector + testbench (Verilog) 23-4
Moore FSM, 2 states: EVEN (out 1, reset/start state) and ODD (out 0). input 1 toggles state, input 0 keeps it. So output = 1 at the start (zero 1s = even).



```
module even_ones(input clk, rst, x,
output reg y);
parameter EVEN = 1'b0, ODD = 1'b1;
reg state, next;
always @(posedge clk or posedge rst)
if (rst) state <= EVEN;
else state <= next;
always @(*) begin
case (state)
EVEN: next = x ? ODD : EVEN;
ODD: next = x ? EVEN : ODD;
endcase
y = (state == EVEN); // 1 at start
end
endmodule
```

```
module tb;
reg clk = 0, rst = 1, x = 0; wire y;
even_ones dut(clk, rst, x, y);
always #5 clk = ~clk;
initial begin
$monitor("%0t x=%b y=%b", $time, x, y);
#12 rst = 0;
x = 1; #10 x = 0; #10 x = 1; #10 x = 1;
#10 x = 0; #10 x = 1; #10 $finish;
end
endmodule
```

#28 NOPs, forwarding speed-up, code scheduling 22-6
(a) No fwd: (n+0.4n)×250 = 350n ps. Fwd: (n+0.05n)×300 = 315n ps. Speed-up = 350/315 = 1.11.
(b) Let x = NOP fraction without forwarding (forwarded version still 0.05n). Need (1+x)·250 > 1.05·300 = 315 ⇒ 1+x > 1.26 ⇒ x > 26% NOPs. (If NOPs scale proportionally, x/8 with fwd: 250(1+x) > 300(1+x/8) ⇒ x > 23.5%).

(c) Assume RF written 1st half / read 2nd half ⇒ dependent instr must be ≥3 slots after producer (2 NOPs if adjacent).

(1) with NOPs	(2) best schedule
add \$s3,\$s1,\$s0	lw \$s3,\$s1,\$s0
nop	add \$s1,0(\$s4)
nop	nop
lw \$s3,4(\$s3)	lw \$s3,4(\$s3)
lw \$s1,0(\$s4)	nop
nop	or \$s2,\$s3,\$s2
or \$s2,\$s3,\$s2	nop
nop	\$s2
nop	sw \$s2,0(\$s3) ; 5 NOP
sw \$s2,0(\$s3) ; 5 NOP	

add→lw→or→sw is a true RAW chain (each needs 3 slots ⇒ sw at slot ≥10 ⇒ 10 slots - 5 instrs = 5 NOPs minimum). Only lw \$s1 is independent; it can't move above add (WAR: add reads \$s1). So NOPs cannot go below 5 without forwarding (with forwarding: 0 NOPs needed).